The 7th Annual Parallel Applications Workshop, Alternatives to MPI+X

Intel® SHMEM: GPU-initiated OpenSHMEM using SYCL

Alex Brooks, Philip Marshall, David Ozog, Md. Wasi-ur- Rahman, Lawrence Stewart, Rithwik Tom

intel

Notices & Disclaimers

All information provided here is subject to change without notice. Contact your Intel representative to obtain the latest Intel product specifications and roadmaps. Intel provides these materials as-is, with no express or implied warranties.

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex. Performance results are based on testing as of dates shown in configurations and may not reflect all publicly available updates. See backup for configuration details. No product or component can be absolutely secure. See backup for configuration details. For more complete information about performance and benchmark results, visit www.intel.com/benchmarks

This document contains information on products in the design phase of development which Intel may change at any time without notice. Do not finalize a design with this information. Results have been estimated or simulated using internal Intel analysis or architecture simulation or modeling and provided to you for informational purposes. Any differences in your system hardware, software or configuration may affect your actual performance. For more complete information about performance and benchmark results, visit www.intel.com/benchmarks.

Intel® , Intel logo and Xeon® are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the U.S. and/or other countries. Other names and brands may be claimed as the property of others.

Intel technologies' features and benefits depend on system configuration and may require enabled hardware, software or service activation. Performance varies depending on system configuration. No computer system can be absolutely secure. Check with your system manufacturer or retailer or learn more at [intel.com].

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

WARNING - This document contains technical data whose export is restricted by the Arms Export Control Act (Title 22, U.S.C., Sec 2751, etq.) or the Export Administration Act of 1979, as amended, Title 50, U.S.C., App. 2401 et seq. Violations of these export laws are subject to severe criminal penalties. Disseminate in accordance with provisions of DoD Directive 5230.25.

Motivation

- Distributed multi-GPU systems are growing rapidly
- 9 of the top 10 TOP500 systems include GPU accelerators
  - #1 – Frontier has 4 GPUs per node
  - #2 – Aurora has 12 Ponte Vecchio (PVC) GPU tiles on each node connected with Intel Xe Link Fabric and with dual Sapphire Rapids (SPR) CPUs
  - #3 – Eagle has 8 GPUs per node
- Current OpenSHMEM standard does not yet enable GPUs
- Vendor specific solutions exist that support specific hardware and programming model

https://singularityhub.com/2022/05/30/age-of-exascale-wickedly-fast-frontier-supercomputer-ushers-in-the-next-era-of-computing/

https://www.datacenterdynamics.com/en/news/doe-finally-starts-installing-intels-oft-delayed-aurora-supercomputer-now-expected-to-hit-2-exaflops/

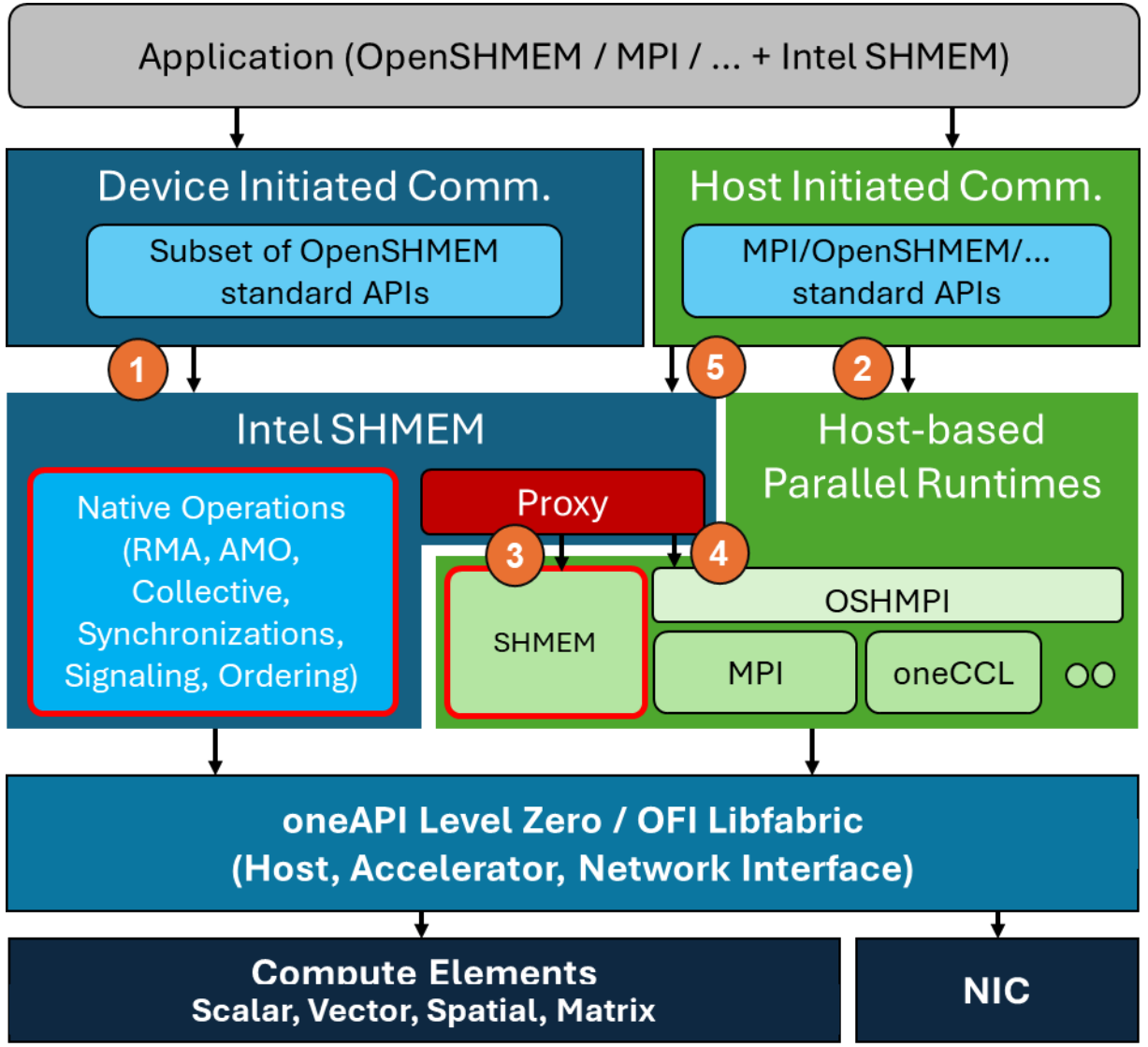
https://www.datacenterdynamics.com/en/news/microsoft-reveals-immense-cloud-supercomputer-just-openai/

Contribution

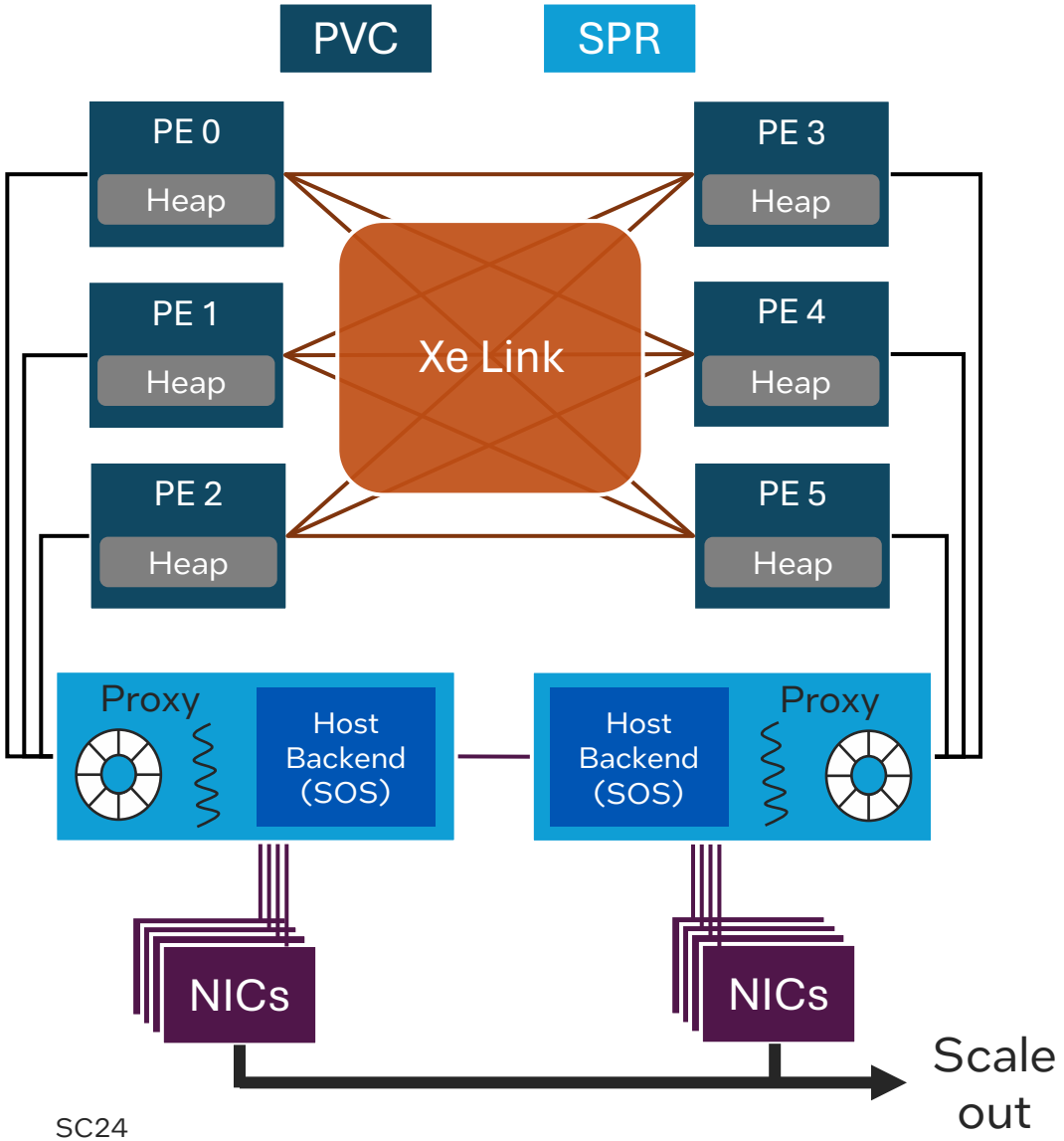
- Intel® SHMEM
  - A library for one-sided and collective communication
  - Implements the OpenSHMEM API, with extensions for GPU using SYCL
- Operations Supported
  - Remote memory access (RMA), atomic memory operations, collective operations, synchronization of multiple processing elements or PEs (i.e., MPI ranks), signaling operations – aligned with OpenSHMEM 1.5
  - API available on host and inside SYCL kernels (device initiated)
- Benefits
  - Simplified programming model for GPU-initiated operations
  - High-performance communications for distributed GPU applications
  - Can leverage high-performance host runtimes

Architecture Overview

1. Device-initiated OpenSHMEM operations
   - Subset of the standard APIs
   - Extensions to leverage GPU threads
   - Intra-node operations leverage L0 IPC for direct GPU to GPU transfer
2. All host back-end operations are supported as it is
3. Inter-node device-initiated operations utilize a host proxy and SHMEM runtime
4. OSHMPI can be utilized to leverage other runtimes (i.e., MPI) [out of scope]
5. Host-initiated operations on GPU memory



Design and Implementation



- Each processing element (PE) is mapped to a single PVC tile (for simplicity, PEs are mapped to whole device)
- Symmetric heap is allocated on each device tile
  - Allocated device heaps are registered to host back-end by extension APIs
  - Sandia OpenSHMEM (SOS) is used as the host back-end
- Intra-node inter-GPU transfers utilize the Xe Links for direct load / store
- Synchronization with host is achieved by a CPU proxy thread via a reverse offload ring buffer
  - 20 million requests per second with multiple GPU threads and a single CPU proxy
- Inter-node transfers use GPU RDMA using FI_HMEM capabilities of supported OFI providers

Device Extension APIs

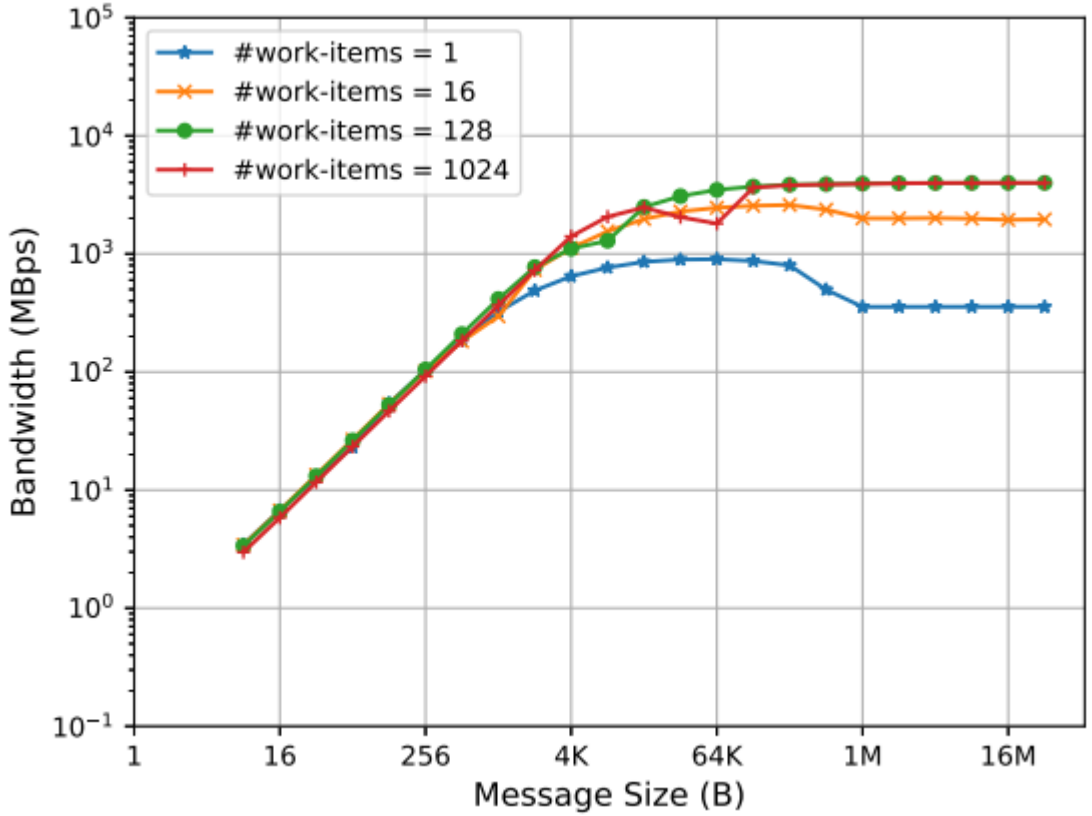
- GPUs launch group of threads to exploit the underlying SIMD/SIMT architecture
- We propose extensions that are callable from GPU kernel only

```
template<typename TYPE, typename Group>
void ishmemx_put_work_group(TYPE *dest, const TYPE *source, size_t nelems,
                            int pe, const Group &group)
```

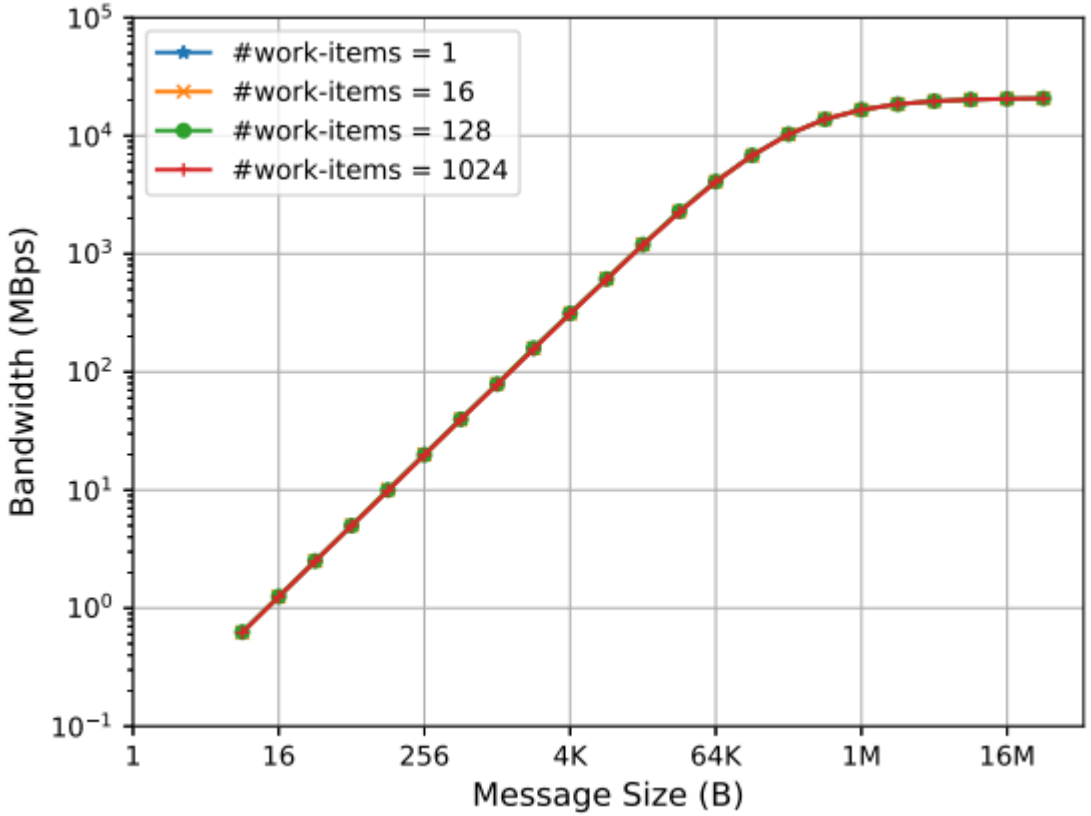
```
template<typename TYPE, typename Group>
int ishmemx_broadcast_work_group(ishmem_team_t team,
                                 TYPE *dest, const TYPE *source, size_t nelems,
                                 int pe, const Group &group)
```

- Requires a Group argument, which corresponds to SYCL work-group/sub-group
- Collective across work-items/threads in a Group
- For intra-node operations, work-items/threads in the Group can execute independently and in any order within the GPU, which optimizes performance
- For inter-node operations, a single designated thread initiate reverse offload, reducing contention

Store v/s Copy Engines (`ishmemx_put_work_group`)



Store from kernel work items



Copy engine transfer

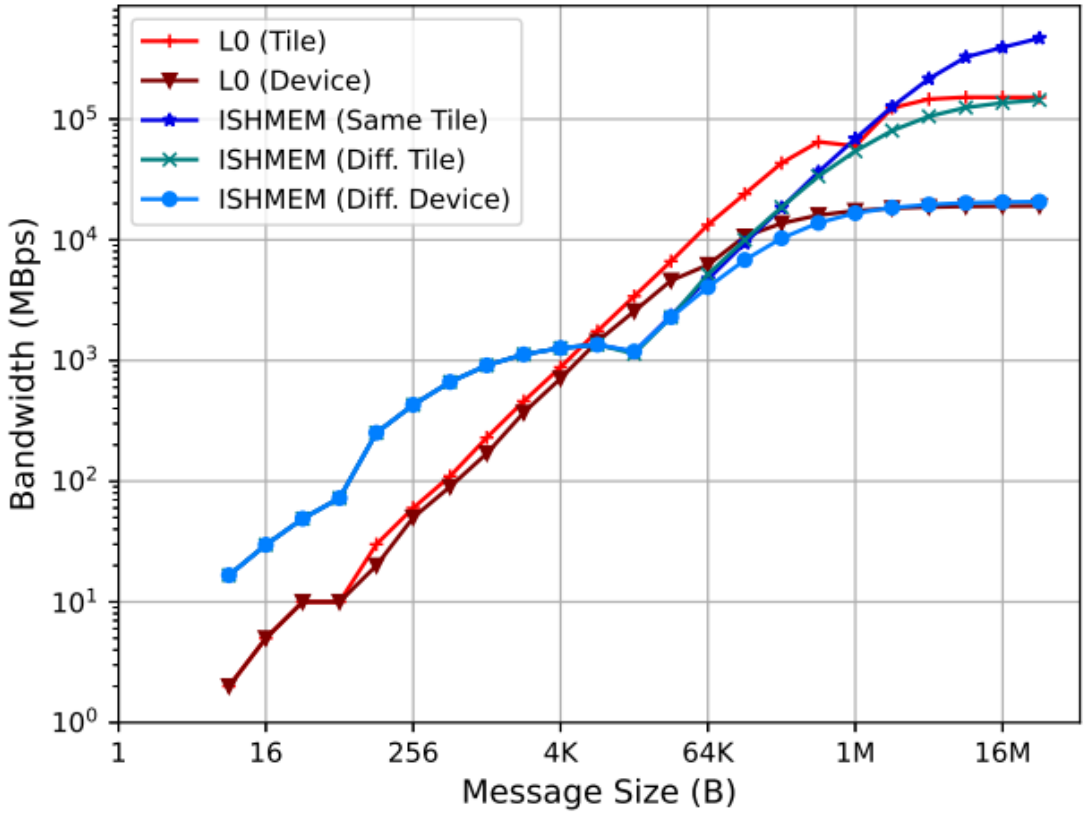
- Kernel driven direct load/store operation performs well for small to medium message sizes
- GPU copy engines provide higher bandwidth for larger message sizes
- Choosing appropriate cutover for performance depends on both message size and number of work-items

See slide #7 for workloads and configurations. Results may vary.

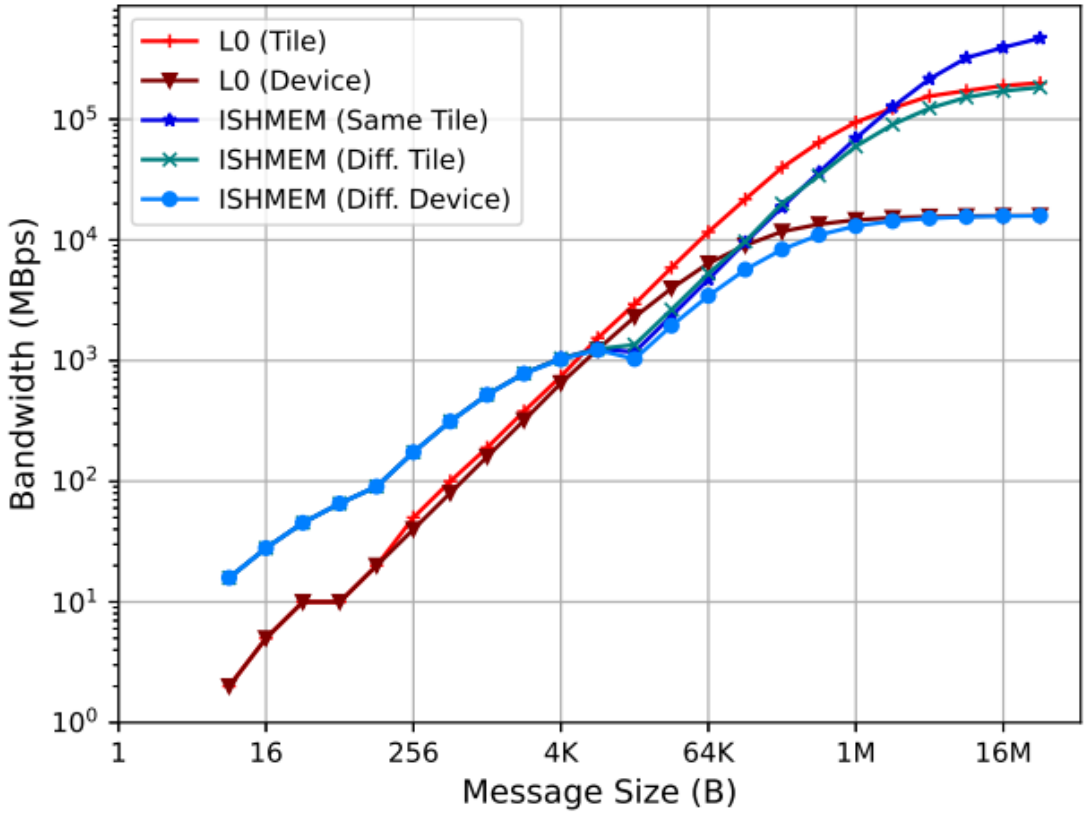
Experimental Setup

- Measurements done on Intel internal Aurora validation system, Borealis
- Hardware details:
  - 6 Intel® Data Center GPU Max Series (PVC) devices fully connected by Xe Link fabric
  - 2x Intel® Xeon® CPU Max 9470C (SPR) CPUs
    - 2.0 GHz, each socket with 52 cores and 2 threads per core
  - 8x HPE* Slingshot 11 NICs (200 Gbps each)
- OS: SUSE* Linux Enterprise Server 15 SP4
- Software Details:
  - Intel® SHMEM 1.1.0
  - Sandia OpenSHMEM (main branch)
  - Intel® oneAPI DPC++/C++ Compiler 2024.1.0
- Measurements are done using Intel® SHMEM performance benchmarks on August, 2024
- Performance results are replicated on Aurora Sunspot system

Performance – RMA (Standard API)



Device-initiated Put Bandwidth

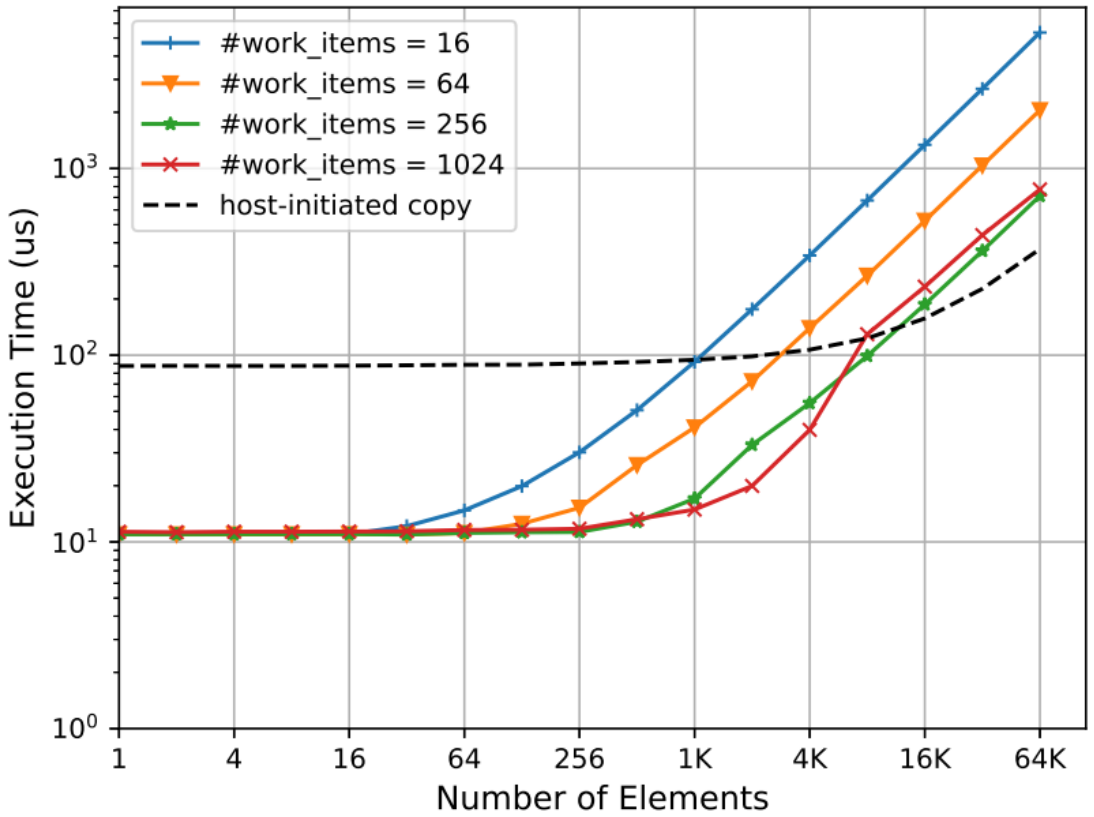


Device-initiated Get Bandwidth

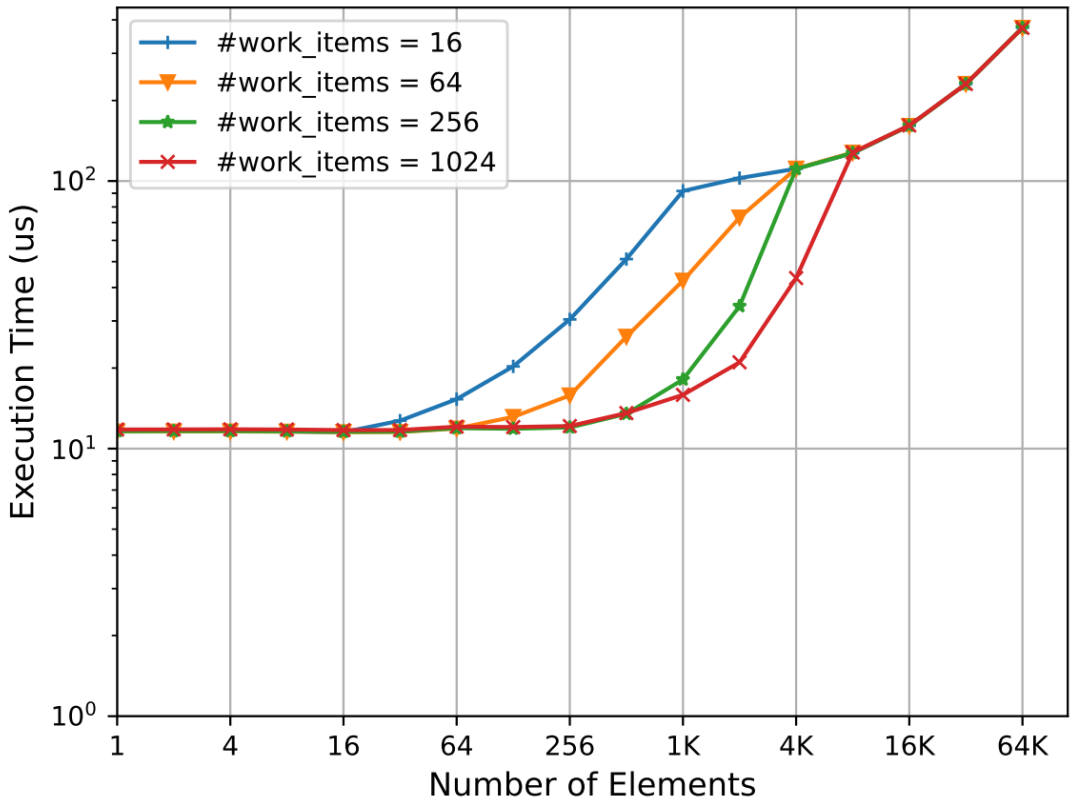
- Device-initiated operations perform better for up to 4KB message size; For larger message sizes, performance is similar to L0

See slide #7 for workloads and configurations. Results may vary.

Performance – Collective (Device Extension API)



Fcollect WG (12 PEs) – w/o cutover tuning



Fcollect WG (12 PEs) – w/ cutover tuning

- Number of work-items per work-group and total number of PEs affect the collective performance; Tuning for cutover ensures best performance across message sizes

See slide #7 for workloads and configurations. Results may vary.

Conclusion

- Intel® SHMEM 1.2.0 is available now
  - https://github.com/oneapi-src/ishmem
- GPU and Host initiated OpenSHMEM operations on GPU memory
- Optimized performance achieved by choosing appropriate cutover points between device driven load/store and host copy engine transfers
- Small operations are low latency; large operations achieve full hardware bandwidth

Intel® SHMEM: Scaling GPU Applications Up and Out

Dev Hub Tutorial at Intel Booth on Nov 20 10:30 AM


intel